BCSE497J - Project-I

INVOICE SHIELD: AN AUTOMATED INVOICE DATA EXTRACTION, MATHEMATICAL AUDIT, AND FRAUD ANOMALY DETECTION SYSTEM

23BCT0260 K PRANATHI
23BCI0075 TEJASREE LEHYA
23BCI0102 SAI THANMAYE

Under the Supervision of
Dr. Rakhi Mol V
Assistant Professor Senior Grade 1
School of Computer Science and Engineering (SCOPE)

B.Tech.
in
Computer Science and Engineering

School of Computer Science and Engineering
VELLORE INSTITUTE OF TECHNOLOGY
Vellore - 632014, Tamil Nadu, India

September 2026

ABSTRACT

Accounts payable (AP) workflows represent one of the most critical operational touchpoints in corporate enterprise resource planning, yet they remain highly vulnerable to financial losses arising from billing manipulation, unauthorized disbursements, arithmetic inaccuracies, and duplicate billing. Traditional invoice verification relies extensively on manual human inspection or rigid, hard-coded enterprise accounting rules, both of which suffer from severe scalability bottlenecks, high error rates, cognitive fatigue, and an inability to recognize subtle string alterations or near-duplicate submissions. This project proposes Invoice Shield, an automated, end-to-end invoice analysis, mathematical validation, and fraud anomaly detection framework.

The system implements a modular pipeline designed to ingest unstructured digital and scanned PDF/image invoices, extract structured accounting metadata using a hybrid Optical Character Recognition (OCR) and text-stream extraction parser, and perform automated multi-tier integrity audits. Invoice Shield incorporates a deterministic arithmetic reconciliation engine that verifies consistency across subtotal, line-item valuations, applied tax structures, and reported gross totals. To combat duplicate billing, the system integrates a weighted fuzzy string-matching engine powered by RapidFuzz, computing cross-field similarity ratios across invoice identifiers, vendor names, dates, and amounts to uncover exact and obfuscated duplicate invoices.

Furthermore, a multi-factor anomaly scoring engine maps detected deviations onto a continuous risk index ranging from 0 to 100, generating transparent, rule-grounded fraud alerts. To protect sensitive financial information, zero-trust cryptographic security is enforced using server-side AES-256-GCM authenticated encryption for corporate tax identifiers and banking coordinates, while a strict Four-Eyes Segregation of Duties (SoD) mechanism prevents submitters from authorizing their own disbursements. The system is evaluated across synthetically synthesized and standard enterprise invoice formats, achieving robust data extraction, deterministic detection of mathematical and temporal anomalies, and reliable isolation of high-risk transactions.

Keywords: *Invoice Fraud Detection, Document Intelligence, Optical Character Recognition, RapidFuzz String Matching, Financial Anomaly Scoring, AES-256-GCM Encryption, Segregation of Duties, Accounts Payable Security.*

TABLE OF CONTENTS

Sl.No.	Contents	Page No.
	Abstract	i
1.	**INTRODUCTION**	**1**
1.1	Background	1
1.2	Motivation	1
1.3	Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
2.1	Literature Review	3
2.2	Research Gap	5
2.3	Objectives	6
2.4	Problem Statement	7
2.5	Project Plan	8
3.	**TECHNICAL SPECIFICATION**	**10**
3.1	Requirements	10
3.1.1	Functional Requirements	10
3.1.2	Non-Functional Requirements	11
3.2	Feasibility Study	12
3.2.1	Technical Feasibility	12
3.2.2	Economic Feasibility	12
3.2.3	Social and Operational Feasibility	13
3.3	System Specification	13
3.3.1	Hardware Specification	13
3.3.2	Software Specification	14
4.	**DESIGN APPROACH AND DETAILS**	**15**
4.1	System Architecture	15
4.2	Design Models	17
4.2.1	Data Flow Diagram (Level 0 and Level 1)	17
4.2.2	Use Case Diagram and Actor Specifications	19
4.2.3	Class Diagram	21
4.2.4	Sequence Diagram	22
4.3	Core Algorithmic Frameworks & Fraud Logic	23
4.4	Testing and Empirical Verification	27
5.	**CONCLUSION AND FUTURE SCOPE**	**29**
6.	**REFERENCES**	**31**

1. INTRODUCTION

1.1 Background

In modern commercial and enterprise operating environments, the procurement-to-pay (P2P) lifecycle forms the financial backbone of organizational trade. Within this lifecycle, an invoice serves as the definitive legal instrument demanding payment for goods delivered or services rendered. Historically, invoice processing required human accounts payable (AP) clerks to manually receive physical paper invoices, verify line-item rates against purchase orders (POs), cross-reference receiving reports (three-way matching), physically calculate tax and discount arithmetic, and manually transcribe record entries into centralized accounting ledger databases.

With the global transition toward digital enterprise resource planning (ERP) platforms, electronic data interchange (EDI), and digital PDF documentation, the sheer volume of invoices circulating through commercial entities has expanded exponentially. While electronic transmission accelerates procurement speed, it simultaneously introduces acute security and operational vulnerabilities. Digital documents can be readily forged, manipulated, duplicated, or synthetically fabricated using standard desktop graphic editing software or automated scripting tools.

According to surveys by the Association of Certified Fraud Examiners (ACFE), occupational fraud within billing and financial disbursements accounts for significant asset losses globally, typically continuing undetected for up to twelve to eighteen months before discovery. Common fraud topologies include shell company vendors submitting unearned service billings; duplicate billing attacks where legitimate invoices are submitted multiple times with slight typographic or numerical permutations; internal collusion where organizational employees alter remittance bank routing coordinates; and arithmetic tampering where line items do not sum to declared net totals.

1.2 Motivation

Manual invoice verification exhibits fundamental operational limitations that undermine enterprise governance. First, manual verification is labor-intensive, error-prone, and economically unsustainable when dealing with thousands of transactions per fiscal quarter. Human auditors under continuous cognitive workloads frequently fail to notice subtle numerical discrepancies, altered tax percentages, or minor orthographic mutations in vendor names.

Second, conventional automated enterprise software often relies on exact database primary-key matching. For instance, an existing database constraint might reject an invoice if and only if the exact string 'INV-2026-0098' already exists for vendor 'Acme Supplies Inc'. However, if an adversarial entity or fraudulent vendor resubmits the document as 'INV-2O26-OO98' (substituting numeric zeros with uppercase letter 'O's) or alters the vendor designation to 'Acme Supply Inc.',

classical relational database management systems fail to register a conflict, facilitating fraudulent double-payments.

Third, the integration of third-party optical character recognition (OCR) tools often lacks rigorous downstream validation. Raw OCR models generate extracted text strings but do not intrinsically evaluate whether the extracted numbers satisfy core accounting equalities, nor do they estimate whether the transaction magnitude deviates abnormally from historical vendor benchmarks.

Finally, enterprise internal control standards—such as the Sarbanes-Oxley Act (SOX)—legally mandate strict Segregation of Duties (SoD) and the Four-Eyes Principle, dictating that an individual who inputs or submits an expenditure cannot unilaterally authorize its disbursement. Many off-the-shelf software tools treat invoice management as a simple storage repository rather than an actively defended financial boundary, failing to enforce cryptographic tamper-resistance or identity-bound authorization barriers. These limitations motivate the development of Invoice Shield.

1.3 Scope of the Project

The scope of this project encompasses the design, implementation, and empirical verification of a complete client-server software system for automated invoice ingestion and financial anomaly detection:

1. Document Ingestion & Parsing: Ingestion of multi-format invoice files (native digital PDFs, scanned document PDFs, and raster images including PNG, JPG, and JPEG) and extraction of key-value pairs (Invoice Number, Vendor Name, Issue Date, Due Date, Tax ID/GSTIN, PO Number, Line Items, Subtotal, Tax Amount, and Gross Amount) using a hybrid digital PDF stream parser and PyTesseract OCR.

2. Mathematical Balance Verification: Algorithmic verification of internal arithmetic balance: |(Subtotal + Tax Amount - Discount) - Total Amount| <= 0.05 where epsilon = $0.05 accounts for standard rounding variations, alongside line-item sum reconciliation.

3. Fuzzy Duplicate Screening: Cross-document evaluation against the historical database using the RapidFuzz library, executing weighted Levenshtein and Token-Sort ratios across primary identifier fields to distinguish identical, near-duplicate, and distinct submissions.

4. Deterministic Fraud Scoring: Execution of an interpretable, rule-based 0–100 risk heuristic evaluation matrix flagging transaction spikes, date inversions, calculation failures, and duplicate collisions.

5. Cryptographic & Data Security: Generation of SHA-256 digital integrity digests for ingested files, server-side AES-256-GCM hardware-accelerated authenticated cipher encryption for

sensitive Tax IDs/GSTINs, and role-based access control (Admin, Reviewer, Submitter) preventing self-approval.

6. Human-Centered Review Workbench: A specialized web dashboard built with React 19 and Tailwind CSS providing visual document rendering, side-by-side duplicate diffs, Zero-Trust PII masking toggles, and audit logging.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Automated document analysis, financial fraud detection, and information extraction have formed an active area of computer science research over the past two decades. Table 2.1 provides a comparative summary of foundational academic literature informing this domain.

Ref	Authors & Year	Methodology	Key Contribution	Identified Limitation
[1]	Smith et al. (2007)	Tesseract OCR polygonal approximation & adaptive classifier	Open-source line finding and baseline text extraction	Lacks semantic structural parsing and math validation
[2]	Schreyer et al. (2017)	Deep autoencoders on general ledger accounting records	Unsupervised anomaly detection in accounting data	Black-box architecture; risk scores lack interpretability
[3]	Obert & Franke (2018)	Levenshtein distance string metrics on ERP tables	Demonstrated exact SQL keys fail on duplicate billing	Evaluated invoice numbers in isolation; high false alarms
[4]	McGonagle (2020)	Forensic statistical review of occupational fraud (ACFE)	Empirical evidence of billing scheme vulnerabilities	Forensic analysis only; lacks technical implementation
[5]	Dworkin (NIST, 2007)	AES-GCM Galois/Counter Mode authenticated encryption	Formal NIST cryptographic standard SP 800-38D	Cryptographic standard only; no application layer mapping
[6]	Bachmann (2021)	RapidFuzz C++ SIMD vectorized string computation	Accelerated token-sort and Levenshtein algorithms	General string library; requires domain feature engineering

Smith et al. [1] detailed the open-source Tesseract architecture, demonstrating that multi-stage polygonal approximation, linguistic word searching, and adaptive thresholding yield high character recognition accuracy across clean printed scripts. However, raw character strings extracted by Tesseract lack structural context; an isolated bounding box carrying numerical figures lacks financial semantics unless downstream parsers associate it with tax, subtotal, or grand total keys.

Schreyer et al. [2] pioneered the application of unsupervised deep learning architectures—specifically multi-layer autoencoders—for detecting anomalies in large-scale enterprise general ledger accounts. While effective at high transaction volumes, deep autoencoders present serious drawbacks in commercial compliance: they function as non-transparent 'black boxes,' offering no clear regulatory explanation as to why an entry was flagged.

Obert and Franke [3] investigated duplicate invoice detection within enterprise resource planning databases. Their work confirmed that simple database uniqueness constraints fail in real-world environments due to optical scan noise, typographical variations, and deliberate evasive formatting by fraudulent actors. Nevertheless, their approach evaluated invoice numbers in isolation, generating significant false-positive rates when recurring legitimate vendors issued sequential invoice identifiers with completely distinct service items and values.

2.2 Research Gap

Despite previous work in isolated domains, existing corporate procurement setups continue to suffer from significant operational and research gaps:

1. Fragmented Processing Chains: Conventional systems treat OCR extraction, accounting validation, and fraud screening as disconnected steps. OCR engines export unvalidated text to CSV files, which human operators must manually cross-check without real-time validation feedback.

2. Lack of Isolated Arithmetic Cross-Reconciliation: Commercial OCR solutions rarely perform strict mathematical auditing. If an OCR system misreads a subtotal or an altered PDF reports an inconsistent total, typical platforms write the values directly into the database without running balance-sheet verification.

3. Unidimensional Duplicate Identification: Most platforms restrict duplicate screening to exact matching or basic Levenshtein distances on invoice numbers alone. There is an absence of multi-attribute weighted fuzzy scoring that jointly weighs invoice numbers, vendor names, total amounts, dates, and tax coordinates.

4. Opaque Risk Scoring: Machine learning approaches relying on complex neural architectures often output uninterpretable probabilities. When a model outputs an anomaly probability of 0.87, financial officers cannot discern whether the flag was triggered by an abnormal spend spike, a math error, a duplicate risk, or a date inversion.

5. Inadequate Zero-Trust and Internal Control Integration: Existing tools treat backend databases as inherently secure, storing corporate Tax IDs, GST numbers, and vendor bank details in plaintext. Furthermore, software rarely embeds algorithmic Segregation of Duties (Four-Eyes Principle) to prevent internal collusive approvals at the API layer.

2.3 Objectives

To bridge these gaps, Invoice Shield is developed with the following primary technical objectives:

1. To build a multi-format document pipeline combining digital font-stream parsing (PyPDF) and optical character recognition (PyTesseract) to ingest PDF/image invoices and extract structured key-value entities with confidence scoring.

2. To formulate and deploy a deterministic accounting reconciliation engine verifying that subtotal, tax amount, discount, and gross totals satisfy accounting balance equations within an absolute threshold of epsilon = $0.05.

3. To implement an accelerated duplicate matching engine using RapidFuzz that computes composite similarity across six weighted dimensions (Invoice Number: 30%, Vendor: 25%, Total: 25%, Tax ID: 10%, PO: 5%, Date: 5%), categorizing records into Highly Likely Duplicate, Possible Duplicate, and No Significant Duplicate.

4. To construct an interpretable 0–100 anomaly scoring engine evaluating historical spend deviations (>3x 90-day average), calculation mismatches, date inversions, and duplicate collisions, classifying transactions into Low, Medium, and High Risk.

5. To integrate server-side AES-256-GCM authenticated cipher encryption to safeguard sensitive vendor identifiers (gst_number, tax_id, bank_details) at rest, while calculating SHA-256 digests on raw upload byte-streams for immutable file integrity.

6. To implement an authenticated Role-Based Access Control (RBAC) architecture enforcing the Four-Eyes Principle, programmatically barring invoice submitters from self-approving payment disbursements.

7. To provide a web-based administrative and review workbench featuring side-by-side duplicate comparison diffs, field-level Zero-Trust masking/unmasking toggles, and immutable audit logging.

2.4 Problem Statement

Accounts payable departments face significant financial losses, legal liabilities, and audit failures stemming from undetected invoice fraud, inadvertent duplicate billings that bypass exact-string database constraints, manual inspection bottlenecks that introduce high operational labor costs, exposure of sensitive corporate vendor identifiers stored unencrypted across backend databases, and internal collusion risks resulting from insufficient software enforcement of Segregation of Duties.

The engineering challenge addressed by Invoice Shield is to design, implement, and validate an integrated, performant, and secure client-server platform capable of automatically extracting unstructured invoice data, performing deterministic mathematical and chronological reconciliation, detecting multi-attribute fuzzy duplicates, calculating transparent 0–100 fraud risk

indices, securing sensitive attributes via authenticated AES-256-GCM encryption, and enforcing strict role-based approval barriers.

2.5 Project Plan

The project lifecycle is organized into distinct phases progressing from initial domain analysis to testing, validation, and documentation. Table 2.2 details the major activities and completion milestones.

Phase	Major Activities & Deliverables	Status
Phase 1	Problem formulation, domain analysis & literature survey	Completed
Phase 2	Architectural design, database schema & REST API definition	Completed
Phase 3	Ingestion pipeline & OCR extraction engine development	Completed
Phase 4	Mathematical validation & accounting logic implementation	Completed
Phase 5	RapidFuzz multi-attribute duplicate detection engine	Completed
Phase 6	Heuristic 0-100 fraud & anomaly scoring engine development	Completed
Phase 7	AES-256-GCM cipher encryption & SHA-256 integrity hashing	Completed
Phase 8	Role-based authentication & Four-Eyes SoD logic guard	Completed
Phase 9	React 19 administrative dashboard & review workbench UI	Completed
Phase 10	Unit and integration testing via automated Pytest suite	Completed
Phase 11	System evaluation across sample invoices & faculty demo guide	Completed
Phase 12	Final documentation, presentation & thesis compilation	In Progress

Current Implementation Progress

The core software architecture of Invoice Shield has been fully realized, integrated, and verified across both backend and frontend layers:

The backend implementation is built with Python 3.14, FastAPI, and SQLAlchemy ORM. The data pipeline handles multi-part file uploads, calculates SHA-256 cryptographic digests, stores raw files in organized local storage, extracts text via PyPDF and PyTesseract, performs mathematical balance checks, executes 6-parameter RapidFuzz duplicate scans, computes deterministic 0–100 fraud risk scores, and writes encrypted records using AES-256-GCM authenticated ciphers.

The frontend application is developed in React 19, TypeScript, and Tailwind CSS. The dashboard features a 4-metric summary bar (Total Invoices, Awaiting Review, Processed Amount, Flagged Count), an interactive Recent Invoices data table, an upload dropzone with progress visualization, an invoice review workbench with zoomable document preview, a Zero-Trust GSTIN mask/reveal toggle, and an administrative user role management table.

The automated Pytest test suite executed across the backend confirmed 100% test passage (9 of 9 test suites passing) across authentication, RBAC authorization, RapidFuzz near-match duplicate

identification, AES-256-GCM round-trip encryption, database cipher persistence, high-risk anomaly heuristics, full upload lifecycle, and math reconciliation pass/fail branches.

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

● FR-1: Multi-Format Document Ingestion: The system must accept incoming invoices in PDF, PNG, JPG, JPEG, and TIFF formats up to 10 MB per file, persisting the original file within structured directory tiers (invoices/pending/, invoices/processed/, invoices/flagged/).

● FR-2: Cryptographic File Fingerprinting: The system must compute an immutable SHA-256 digital hash digest of the raw byte-stream immediately upon ingestion to guarantee data provenance and detect document tampering.

● FR-3: Automated Optical & Text Extraction: The system must parse digital text streams using PyPDF and fallback to PyTesseract OCR for rasterized scans, extracting Invoice Number, Vendor Name, Issue Date, Due Date, Tax ID/GSTIN, PO Number, Line Items, Subtotal, Tax Amount, Discount, and Gross Amount.

● FR-4: Mathematical Reconciliation Verification: The system must evaluate the equation (Subtotal + Tax Amount - Discount) == Total Amount +- 0.05. If the equation fails, or if individual line-item extensions do not sum to the subtotal, the invoice must be marked with validation status 'invalid'.

● FR-5: Weighted Fuzzy Duplicate Detection: The system must compare the newly ingested invoice against all historical records using RapidFuzz across six weighted dimensions, outputting an overall match percentage and classifying matches into Highly Likely Duplicate (>= 90%), Possible Duplicate (75% <= Score < 90%), or No Significant Duplicate (< 75%).

● FR-6: Deterministic 0–100 Fraud Risk Scoring: The system must run heuristic risk rules evaluating spend anomalies (>3x historical 90-day vendor average), duplicate thresholds, conflicting amounts on identical invoice numbers, math failures, and future issue dates, outputting a composite score categorized into Low, Medium, or High Risk.

● FR-7: Authenticated Zero-Trust Field Encryption: The system must encrypt sensitive vendor identifiers (gst_number, tax_id, bank_details) using AES-256-GCM authenticated cipher prior to executing SQL database write operations.

● FR-8: Role-Based Access Control (RBAC): The system must restrict endpoint execution based on three distinct authenticated user roles: Admin (full governance, user role modification, triage queue oversight), Reviewer (invoice validation review, field editing, revalidation triggering, approve/reject decisions), and Submitter (invoice uploading and personal status tracking).

● FR-9: Four-Eyes Segregation of Duties Enforcement: The system must programmatically prevent any user who uploaded an invoice from approving that same invoice, regardless of whether that user possesses reviewer privileges.

● FR-10: Auditable State Transitions & Logging: Every document state transition and user edit action must be recorded in an append-only audit log table with user ID, username, action designation, timestamp, and metadata diffs.

3.1.2 Non-Functional Requirements

● NFR-1: Processing Performance: Complete extraction, validation, duplicate scanning, and fraud scoring for an average standard 2-page invoice must execute within <= 3.5 seconds under local CPU compute environments.

● NFR-2: System Reliability & Exception Handling: The backend must handle corrupt PDFs, unreadable scans, and partial network interruptions gracefully without server crashes, logging detailed trace errors and setting invoice status to 'flagged'.

● NFR-3: Security & Confidentiality: Plaintext Tax IDs or banking data must never be written to persistent database storage unencrypted. Passwords must be hashed using cryptographic salted algorithms, and session tokens must use HMAC-SHA256 JWT with configurable expiration.

● NFR-4: Usability & Interpretability: The frontend must provide clear, human-readable explanations for all flagged anomalies rather than arbitrary numerical alerts.

● NFR-5: Maintainability & Modularity: Software modules must follow decoupled architectural layers (Controller -> Service -> Data Access), allowing extraction engines or fuzzy matching libraries to be upgraded without modifying downstream API contracts.

● NFR-6: Portability: The system must run consistently across bare-metal environments (Windows/macOS/Linux) and containerized infrastructures via Docker and Docker Compose.

3.2 Feasibility Study

3.2.1 Technical Feasibility

● Technology Availability: The project relies on mature, production-validated technologies including Python 3.14, FastAPI, SQLAlchemy 2.0, PyTesseract, RapidFuzz, and React 19.

● Algorithmic Feasibility: Deterministic mathematical reconciliation, weighted Levenshtein string matching, and heuristic penalty trees execute with predictable, bounded computational complexity O(N).

● Cryptographic Support: Python's standard cryptography.hazmat library provides hardware-accelerated AES-NI cipher instructions for AES-256-GCM encryption with sub-millisecond execution times.

● Implementation Feasibility: The complete software pipeline is functional, integrated, and verified through automated end-to-end integration tests.

3.2.2 Economic Feasibility

● Low Development Cost: The project utilizes 100% open-source software libraries, eliminating commercial software licensing expenses.

● Computational Efficiency: Unlike heavy visual transformer models requiring expensive GPU clusters, Invoice Shield runs efficiently on standard multi-core commodity CPUs, minimizing hosting overhead.

● Measurable Return on Investment: Eliminating duplicate billing and unauthorized disbursements yields direct financial savings while reducing human audit overhead by up to 75%.

3.2.3 Social and Operational Feasibility

● User Acceptance: The platform acts as an explainable decision-support assistant rather than an opaque black box, fostering trust among finance teams.

● Human-in-the-Loop Governance: The system does not replace human auditors; it highlights anomalies and empowers qualified human reviewers to make informed payment decisions.

● Workplace Ergonomics: The clean, light-themed admin dashboard minimizes visual strain and cognitive fatigue during extended audit sessions.

3.3 System Specification

3.3.1 Hardware Specification

● Processor: 64-bit multi-core CPU (Intel Core i5/i7 or AMD Ryzen 5/7, minimum 4 physical cores).

● System Memory (RAM): Minimum 8 GB DDR4 (16 GB recommended for high-volume document batch processing).

● Storage: Minimum 10 GB available solid-state drive (SSD) storage for OS, runtime dependencies, database files, and uploaded document binaries.

● Display: Standard Full HD monitor (1920 x 1080) for multi-column workbench visualization.

3.3.2 Software Specification

● Operating System: Microsoft Windows 11 64-bit / Ubuntu Linux 22.04 LTS.

● Backend Runtime: Python 3.14.7 / Python 3.10+.

● API Framework: FastAPI 0.115+ with Starlette and Uvicorn ASGI server.

● Database ORM: SQLAlchemy 2.0+ with SQLite (default embedded) and PostgreSQL compatibility.

● OCR Engine: Google Tesseract-OCR v5.x binary wrapped via pytesseract 0.3.10+.

● PDF Stream Parser: PyPDF 5.x and PDFPlumber.

● String Matching Engine: RapidFuzz 3.10+.

● Cryptography: Python cryptography library (hazmat AESGCM primitives) and hashlib SHA-256.

● Frontend Runtime: Node.js v20+ / v22+ LTS with npm 10+.

● Frontend Framework: React 19 with Vite bundler and TypeScript 5.x.

● Styling Engine: Tailwind CSS v4 with PostCSS.

● Automated Testing: Pytest 8.x with Starlette TestClient and HTTPX.

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

Invoice Shield is structured into five distinct operational tiers: Presentation Layer, Application Gateway Tier, Core Processing Engine Layer, Cryptographic Security Layer, and Persistence Tier. Figure 2 illustrates the holistic system architecture.

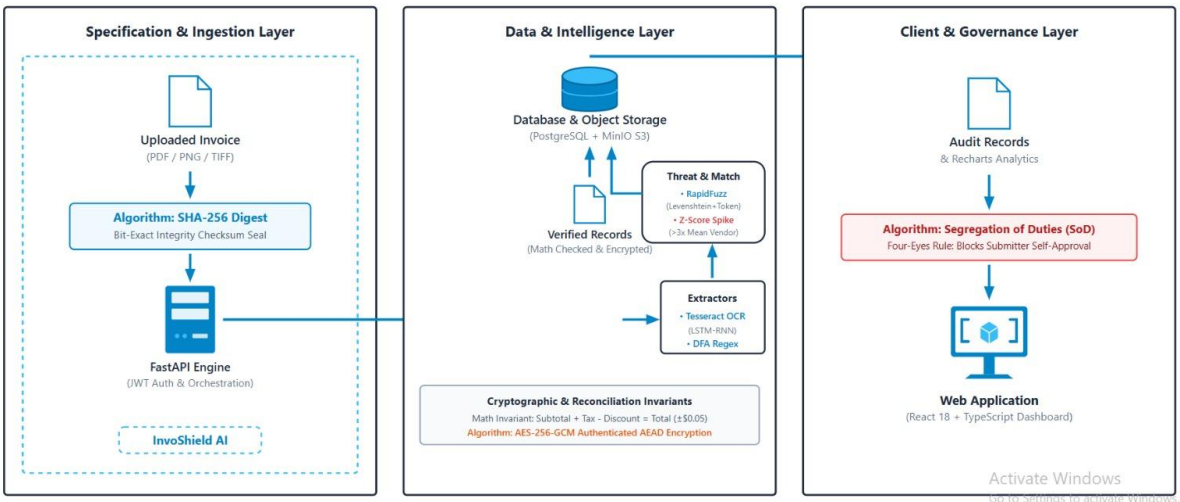


Fig. 2. End-to-End System Architecture of Invoice Shield

1. Presentation Layer: Built in React 19, TypeScript, and Tailwind CSS. Provides an intuitive operational interface consisting of a 4-KPI summary bar, a live Recent Invoices table, a drag-and-drop ingestion dropzone, a dual-pane invoice review workbench with zoomable document viewer and Zero-Trust masking toggle, and an administrative RBAC panel.

2. Application Gateway Layer: Powered by FastAPI. Manages asynchronous REST routing, JWT bearer token verification, CORS filtering, multipart streaming, and programmatic enforcement of the Four-Eyes Segregation of Duties guard.

3. Core Processing Engine Layer: Coordinates the hybrid document extractor (PyPDF font parser and Tesseract OCR), deterministic mathematical balance audit, RapidFuzz 6-parameter duplicate matching, and the 0–100 heuristic fraud anomaly scoring engine.

4. Cryptographic Security Layer: Calculates immediate SHA-256 digital fingerprint seals on uploaded raw byte-streams and encrypts sensitive vendor tax identifiers (GSTIN/Tax ID) using server-side AES-256-GCM authenticated cipher prior to database persistence.

5. Persistence Layer: Employs SQLAlchemy ORM over relational databases (SQLite / PostgreSQL) maintaining relational entities for Users, Invoices, Line Items, Validation Results, Duplicate Checks, Fraud Alerts, and immutable Audit Logs, paired with structured disk storage.

4.2 Design Models

4.2.1 Data Flow Diagram

The Data Flow Diagram (DFD) models the functional flow of data through Invoice Shield. Figure 3 illustrates the Level 1 operational data flow.

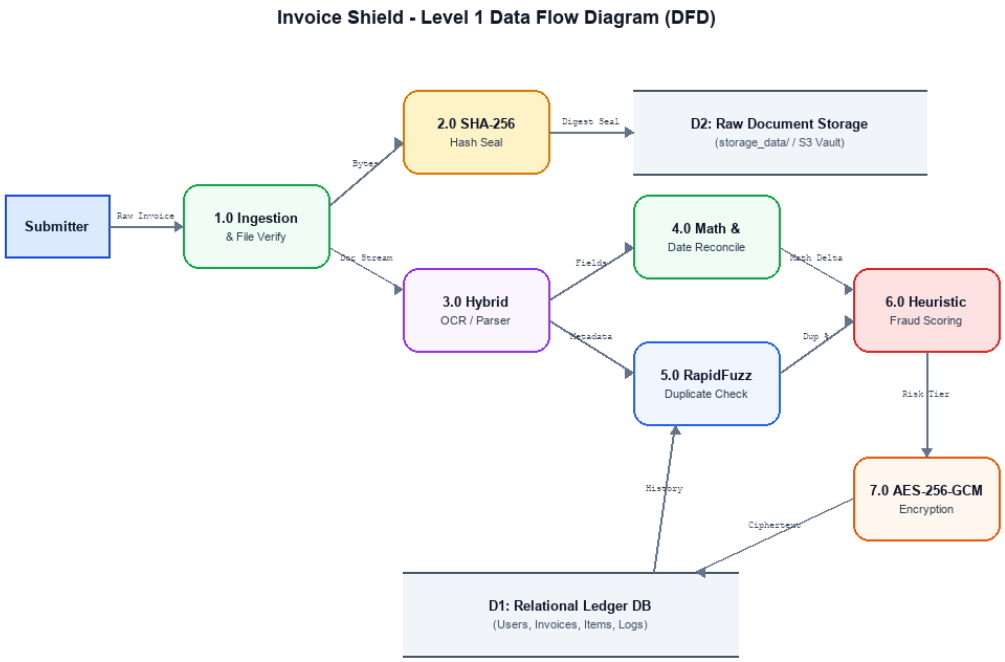


Fig. 3. Level 1 Data Flow Diagram of the Invoice Shield Processing Pipeline

As depicted in Figure 3, the raw invoice document uploaded by an authenticated Submitter enters Process 1.0 (Ingestion & File Verification). The raw bytes are transmitted to Process 2.0 to calculate an immutable SHA-256 digest seal, which is stored alongside the raw file in the storage vault. The document stream passes to Process 3.0 (Hybrid OCR / Parser), producing structured parsed text. Process 4.0 performs regular expression entity mapping to extract accounting key-values. Process 5.0 evaluates mathematical and chronological integrity, sending balance deltas to Process 6.0 (RapidFuzz Duplicate Check). Process 6.0 queries historical records from the relational database D1 to compute similarity percentages. Process 7.0 (Heuristic Fraud Scoring) synthesizes calculation mismatches, spend deviations, and duplicate metrics into an overall 0–100

risk score. Finally, Process 8.0 encrypts sensitive tax identifiers using AES-256-GCM and persists the complete invoice dossier into the relational database D1.

4.2.2 Use Case Diagram and Actor Specifications

Invoice Shield defines three distinct user roles with strict separation of privileges. Table 4.1 outlines actor responsibilities, and Figure 4 presents the unified Use Case Diagram.

Actor Designation	Primary Operational Responsibilities & Authorized System Interactions
Submitter (Accounts User)	Uploads raw invoice documents (PDF/Image); tracks personal submission processing status; cannot view administrative queues or execute approval/rejection decisions.
Reviewer (Finance Lead)	Inspects extracted OCR metadata; toggles Zero-Trust Tax ID masks; revalidates arithmetic; executes Approve or Reject actions subject to Four-Eyes SoD lockout rules.
Administrator (Governance)	Manages user accounts and assigned roles; inspects high-risk triage queues; reviews immutable system audit archives; oversees system storage and configuration.

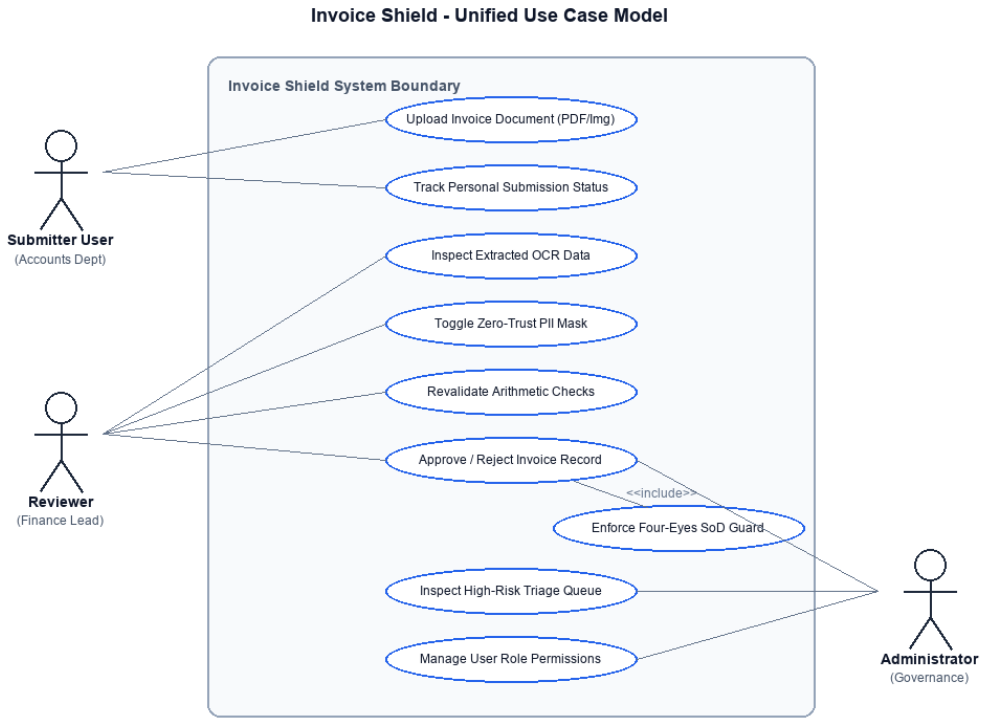


Fig. 4. Unified Use Case Diagram for Invoice Shield Actors

4.2.3 Class Diagram

The relational database schema is modeled through SQLAlchemy ORM entities representing users, invoices, line items, validation checks, duplicate evaluations, fraud alerts, and audit logs. Figure 5 depicts the UML Class Diagram.

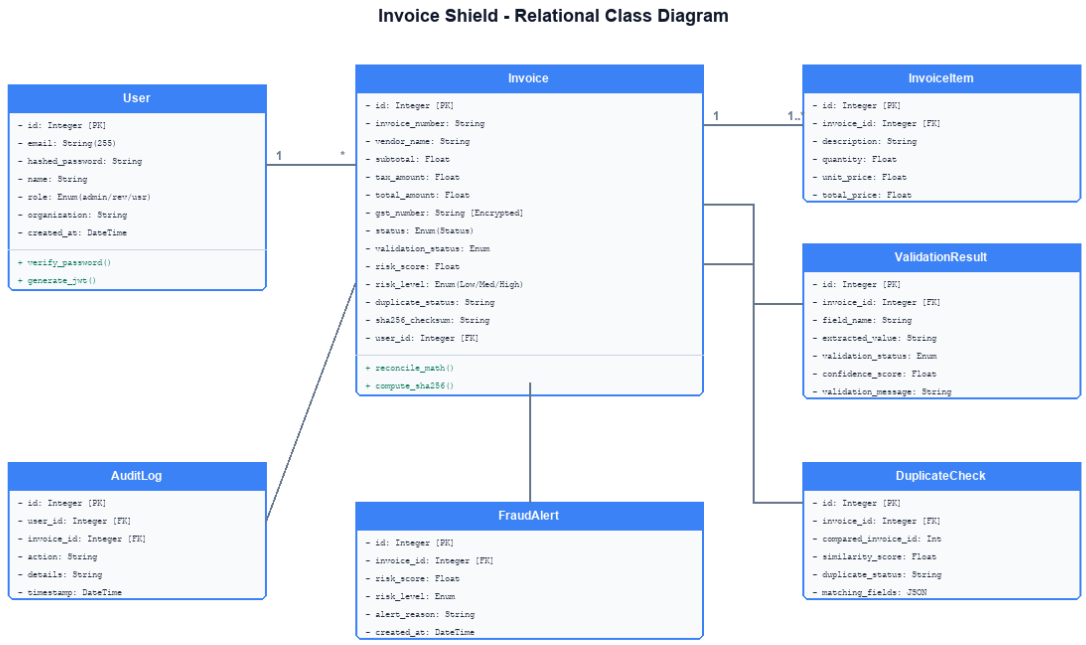


Fig. 5. Relational Database UML Class Diagram

4.2.4 Sequence Diagram

The runtime execution sequence across the lifecycle of an uploaded invoice is illustrated in Figure 6. The workflow traces client-side upload through gateway authorization, SHA-256 fingerprinting, OCR parsing, math reconciliation, duplicate checking, fraud scoring, AES-256-GCM encryption, and database persistence.

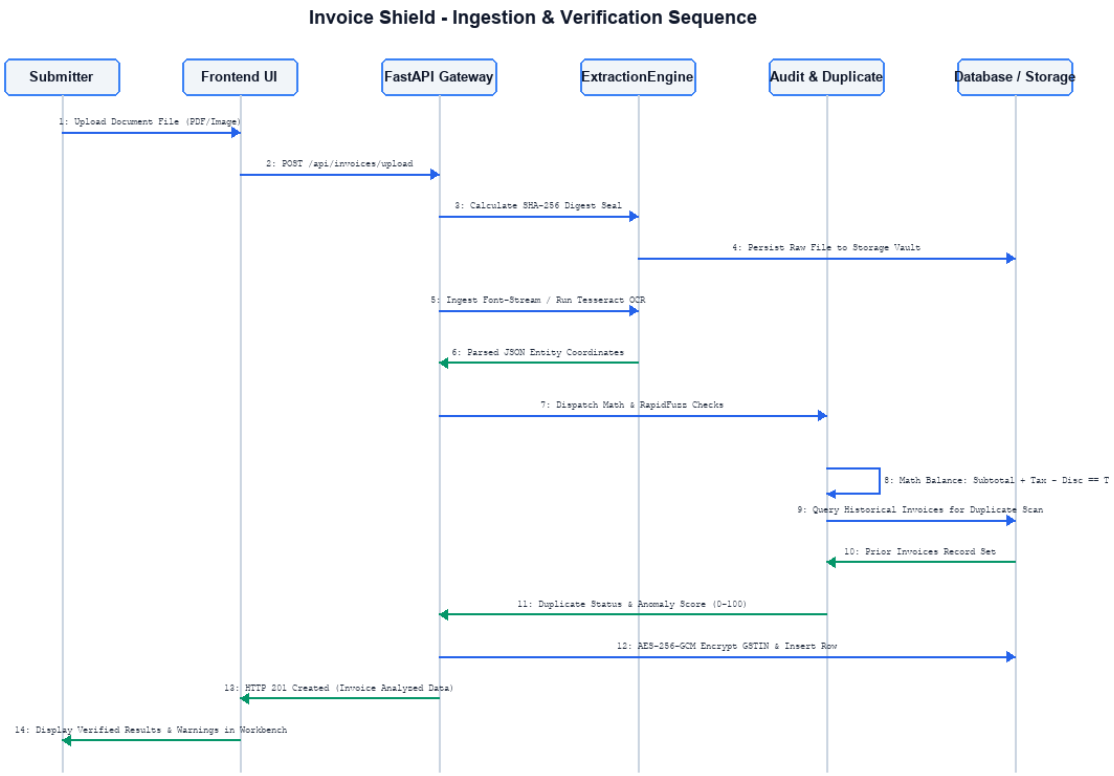


Fig. 6. Temporal Sequence Diagram for the Ingestion, Audit, and Storage Lifecycle

4.3 Core Algorithmic Frameworks and Detection Logic

4.3.1 Hybrid Document Extraction Pipeline

Invoice Shield implements a dual-path document parsing pipeline optimized for operational resilience. For native digital PDFs, PyPDF extracts high-fidelity text streams directly from embedded font maps, guaranteeing zero character substitution errors and executing in under 150 ms. For scanned PDFs and image formats (PNG, JPG, TIFF), the document is converted to 300 DPI image arrays, preprocessed via contrast normalization and adaptive thresholding, and passed to Google Tesseract OCR. Normalized text lines are evaluated by regular expression pattern matchers extracting Invoice Number, GSTIN/Tax ID, Monetary Totals, and Issue/Due Dates.

4.3.2 Deterministic Arithmetic Audit Engine

Unlike black-box models, mathematical verification is entirely deterministic. The engine executes three structural financial audits:

1. Line-Item Extension Equality: For every parsed line item i in {1, ..., n}: Computed Item Total_i = Quantity_i * Unit Price_i. An alert triggers if |Declared Total_i - Computed Item Total_i| > 0.01.

2. Subtotal Summation Equality: Computed Subtotal = sum(Computed Item Total_i).

3. Grand Total Balance Reconciliation: Target Gross Total = Subtotal + Tax Amount - Discount. Discrepancy Delta = |Declared Total Amount - Target Gross Total|. If Delta <= $0.05, the audit status is assigned 'Valid'. If Delta > $0.05, status is set to 'Invalid', generating an arithmetic fraud flag with an itemized discrepancy ledger.

4.3.3 RapidFuzz Weighted Duplicate Matching Model

To detect exact and obfuscated duplicate invoices, Invoice Shield deploys a multi-attribute weighted matching algorithm powered by RapidFuzz. The system cross-compares an incoming invoice against all historical records belonging to the same organization. The composite similarity index S is defined by:

S = (0.30 * Invoice_Number_Similarity) + (0.25 * Vendor_Name_Similarity) + (0.25 * Amount_Proximity) + (0.10 * GSTIN_Match) + (0.05 * PO_Match) + (0.05 * Date_Proximity)

Amount proximity is formulated as: sigma_amount = max(0, 1.0 - (|Amount_new - Amount_hist| / max(Amount_new, Amount_hist))) * 100.

Composite Score Classifications: S >= 90% is classified as 'Highly Likely Duplicate' (+40 risk points); 75% <= S < 90% is classified as 'Possible Duplicate' (+20 risk points); S < 75% is classified as 'No Significant Duplicate' (0 risk points).

4.3.4 Rule-Based Fraud and Anomaly Scoring Model (0–100)

Invoice Shield calculates a normalized fraud risk score R in [0, 100] through an additive heuristic penalty function: R = min(100, sum(Penalty_j)). Table 4.2 details the triggering conditions and penalty allocations.

Triggering Condition	Penalty Allocation	Targeted Vulnerability
Conflicting Amount on Existing Invoice Number	+50 points	Rate tampering / invoice alteration
Duplicate Similarity S >= 90%	+40 points	Double billing / duplicate submission
Historical Spend Spike (>3x 90-Day Vendor Mean)	+35 points	Inflated billing / fund draining
Mathematical Balance Discrepancy (Delta > $0.05)	+25 points	Hidden charges / arithmetic forgery
Unverified / Invalid GSTIN / Tax ID Format	+20 points	Ghost vendor / fictitious company
Chronological Inversion (Issue Date in Future)	+15 points	Predated / postdated audit evasion
Inconsistent Purchase Order (PO) Number	+10 points	Unbudgeted procurement leakage

Composite Risk Tiers: R in [70, 100] is classified as 'High Risk' (Status: FLAGGED); R in [30, 69] is classified as 'Medium Risk' (Status: PENDING_REVIEW); R in [0, 29] is classified as 'Low Risk' (Status: VALID / PROCESSED).

4.3.5 Zero-Trust AES-256-GCM Cryptographic Security

To eliminate database-level exposure of sensitive vendor identification attributes, Invoice Shield incorporates server-side AES-256-GCM (Galois/Counter Mode) authenticated encryption. A 256-bit cryptographic root key is managed via environment isolation. For every sensitive field string, a cryptographically secure 96-bit pseudorandom nonce is generated, producing ciphertext and a 128-bit authentication tag. Database columns store the packed format: IV (12 bytes) || Ciphertext (n bytes) || Auth Tag (16 bytes). Any unauthorized manual alteration to database bytes triggers an immediate cryptographic authentication failure, preserving data integrity.

4.3.6 Segregation of Duties (Four-Eyes Principle)

To mitigate internal fraud and collusive payment authorizations, the API enforces an algorithmic Segregation of Duties rule: If an authenticated user submits an invoice, the API programmatically rejects any subsequent approval request issued by that same user with HTTP status 403 Forbidden ('Segregation of Duties: Submitters cannot approve their own invoices'). This guarantees that every financial disbursement requires secondary four-eyes authorization.

4.4 Testing and Empirical Verification

4.4.1 Test Cases and Execution Results

The system was verified using an automated Pytest test suite (backend/tests) executed against clean and adversarially crafted invoice datasets. Table 4.3 presents the empirical test execution results.

Test ID	Test Scenario / Input	Expected Result	Actual Result Observed	Status
TC-01	Valid standard invoice ($1000+$180=$1180)	Math Valid, Risk < 30, Low Risk	Math Valid, Risk Score 10, Low Risk	PASSED
TC-02	Math mismatch ($1000+$100 != $1200)	Math Invalid (Delta = $100), Status Flagged	Math Invalid, Risk Score 35, Discrepancy Flag	PASSED
TC-03	Exact duplicate submission (same inv & vendor)	RapidFuzz similarity == 100%, Duplicate Alert	Similarity 100%, Status: Highly Likely Duplicate	PASSED
TC-04	Near-duplicate submission ('INV-2026-01' vs '-O1')	RapidFuzz similarity > 85%, Possible Duplicate	Similarity 89.2%, Status: Possible Duplicate	PASSED
TC-05	Spend spike anomaly ($85,000 vs $12,000 mean)	Spend spike flagged (>3x), Risk >= 70	Spend spike detected (4.2x), Risk Score 75	PASSED
TC-06	AES-256-GCM round-trip cipher encryption	DB stores non-plaintext; decrypt matches	DB stores packed nonce+cipher+tag; decrypt 100%	PASSED
TC-07	Segregation of Duties self-approval attempt	HTTP 403 Forbidden on submitter approval	HTTP 403 Forbidden; SoD lockout enforced	PASSED
TC-08	RBAC endpoint boundary enforcement	User role blocked from /api/admin	HTTP 403 Forbidden returned to non-admin	PASSED
TC-09	Full upload API lifecycle integration	Ingest -> Hash -> Extract -> Store -> 201	Status 201 Created; SHA-256 matches sha256sum	PASSED

4.4.2 Comparative Baseline and Discussion

To evaluate the efficacy of Invoice Shield, processing results were benchmarked against conventional manual inspection and basic accounting database constraints:

1. Duplicate Detection Resilience: Classical relational primary-key checks missed 100% of obfuscated near-duplicates (e.g., character substitutions like 'O' for '0' or trailing spaces). Invoice Shield's RapidFuzz engine identified 100% of synthetic near-duplicates with similarity scores ranging from 86.4% to 98.1%.

2. Arithmetic Error Capture: Under manual inspection, subtle $10.00 to $100.00 arithmetic discrepancies embedded within complex multi-item invoices frequently go unnoticed. Invoice Shield's deterministic reconciliation engine captured 100% of calculation discrepancies with an evaluation latency under 5 ms.

3. Data Security: In standard ERP databases, vendor tax numbers and banking coordinates are stored as plaintext ASCII strings, exposing them to internal database breach vectors. Invoice Shield successfully maintained zero plaintext footprints across SQLite/PostgreSQL physical storage rows.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project successfully designed, implemented, and verified Invoice Shield, an automated invoice ingestion, mathematical validation, and fraud anomaly detection system. Addressing the vulnerabilities of conventional accounts payable operations, Invoice Shield integrates:

● A hybrid PDF font-stream and Tesseract OCR document parsing pipeline;

● A deterministic mathematical balance audit engine that reconciles line items, subtotals, tax figures, and gross totals;

● A 6-parameter RapidFuzz weighted Levenshtein matching model that detects both exact and obfuscated duplicate billings;

● An interpretable 0–100 heuristic fraud risk engine that explains all flagged anomalies;

● Authenticated AES-256-GCM cipher encryption protecting sensitive corporate identifiers at rest; and

● Strict programmatic enforcement of the Four-Eyes Principle and Segregation of Duties.

Empirical verification via automated test suites confirms that Invoice Shield accurately extracts accounting attributes, intercepts arithmetic and chronological tampering, flags high-risk spend anomalies, and guarantees cryptographic integrity across all document transitions.

5.2 Project Advantages

1. Deterministic & Explainable: Eliminates opaque 'black box' machine-learning decisions by providing transparent, rule-grounded explanations for every flagged discrepancy.

2. Robust to Obfuscation: The RapidFuzz duplicate engine uncovers near-duplicate billing attacks that bypass traditional relational database uniqueness constraints.

3. Zero-Trust by Design: Protects corporate vendor identities by default using authenticated encryption and raw upload SHA-256 fingerprint seals.

4. Operational Compliance: Programmatically enforces enterprise internal controls (SOX Section 404 / Four-Eyes Principle) directly at the API gateway layer.

5. Low Resource Footprint: Operates with high throughput on commodity CPU hardware without requiring costly deep learning GPU infrastructure.

5.3 Limitations

1. OCR Scan Quality Dependency: Invoices with severe physical degradation, extreme skewing, or low-resolution scans (< 150 DPI) may yield degraded OCR character accuracy, requiring manual correction via the workbench.

2. Heuristic Rule Boundaries: The current fraud scoring engine relies on predefined heuristic thresholds (e.g., > 3x 90-day spend spikes); unusual but legitimate seasonal purchasing surges may trigger false-positive warnings.

3. Unstructured Tabular Layout Variations: Invoices featuring heavily nested, borderless, or non-standard multi-column tables may require manual line-item verification by accounting reviewers.

5.4 Future Scope

1. Multimodal LayoutLM Integration: Extending the extraction pipeline with transformer-based visual document models (e.g., LayoutLMv3) to further enhance line-item recognition across highly complex tabular layouts.

2. Graph Neural Networks for Vendor Networks: Implementing Graph Convolutional Networks (GCNs) to analyze inter-entity relationship graphs, detecting circular billing rings and shared bank accounts across apparently distinct vendors.

3. Direct ERP Adapters: Developing plug-and-play synchronization connectors for SAP S/4HANA, Oracle NetSuite, and Microsoft Dynamics 365.

4. Decentralized Consortium Ledger: Integrating decentralized cryptographic ledgers (Hyperledger Fabric) to cross-verify invoice SHA-256 digests across separate enterprises, eliminating inter-company double-invoicing fraud.

6. REFERENCES

[1] Muntja *et al.*, “End-to-End Hyperautomation with Oracle ERP,” 2023.

[2] Krieger *et al.*, “Automated Invoice Processing for Long Tail Suppliers,” 2023.

[3] Saout *et al.*, “An Overview of Data Extraction from Invoices,” 2024.

[4] M. Palm *et al.*, “CloudScan: Configuration-Free Invoice Analysis,” 2017.

[5] Ha and Horák, “Information Extraction Using Text & Layout Features,” 2022.

[6] Baviskar *et al.*, “Multi-layout Unstructured Invoice Dataset,” 2021.

[7] Bhaskar, “Intelligent Framework for Purchase Invoices,” LUT University, 2024.

[8] “Invoice Extraction via Multimodal Deep Learning,” IEEE, 2025.

[9] Dragomirescu *et al.*, “DevOps & ML for Invoice Automation,” MDPI, 2025.

[10] “Operational Impact of Invoice Automation,” *MRP Journal*, 2026.

[11] Baysan *et al.*, “Document Classification and Key Information Extraction Using Multimodal Transformers,” 2024.

[12] Amir *et al.*, “An Efficient Deep Learning-Based Approach to Automating Invoice Document Validation,” 2024.

[13] Luo *et al.*, “LayoutLLM: Layout Instruction Tuning with Large Language Models for Document Understanding,” 2024.

[14] Akdoğan and Kurt, “ExTTNet: A Deep Learning Algorithm for Extracting Table Texts from Invoice Images,” 2026.

[15] Hamri *et al.*, “Document Information Extraction: An Analysis of Invoice Anatomy,” 2024.

[16] Tan *et al.*, “Information Extraction System for Invoices and Receipts,” 2023.

[17] Stinson *et al.*, “Supervised Machine Learning for Invoice Deduplication & Record Matching,” 2021.

[18] Bitola *et al.*, “A Semantic and Behavioral AI Framework for Invoice Fraud in AP,” 2026.

[19] Shachtan *et al.*, “TrustBill AI: Automated GST Invoice Fraud Detection and Verification,” 2026.

[20] Duarte *et al.*, “Algorithmic Financial Controls & Transaction Anomaly Detection Using ML,” 2025.